

GROUP ML REGRESSION PROJECT

Ames Housing Price Prediction

Predicting SalePrice with SVR, Linear Regression, and KNN

TEAM

Marwan · [Teammate 1] · [Teammate 2]

Ames Housing Dataset · 2026

THE PROBLEM

Predict a home's sale price from its physical and locational features

~2,900

Home sales in Ames, Iowa

~80

Raw features per home

3

Required model types: SVR, Linear Regression, KNN

Work was split across the team by pipeline stage, then merged into one shared notebook and verified end-to-end before final evaluation.

What the data told us before we modeled



SalePrice is right-skewed

A long tail of expensive homes. log1p-transforming the target sharply reduced skew and kurtosis — this justified log-transforming the target for Linear Regression later.



Missing $\neq$ unknown

Most "missing" categorical values meant the feature wasn't present at all (no pool, no basement) — not that data was lost.



Automated cross-check

A ydata-profiling report validated the manual EDA and caught anything the manual pass might have missed.

PREPROCESSING PIPELINE

Split first, fit on training data only

1. Dedup

Full-row duplicates dropped before splitting, so no record leaks across train/val/test.

2. Split 60/20/20

Train / validation / test, random_state=42, done before any statistic is computed.

3. Impute + Engineer

Medians/modes learned from train only. 11 new features (Total SF, House Age, etc).

4. Encode + Scale

Yeo-Johnson, one-hot, ordinal encoding — all fit inside the model pipeline.

5. Model

SVR, Linear Regression, and KNN each share the exact same fitted pipeline.

The first version of this pipeline computed statistics on the full dataset before splitting — a leakage bug caught and fixed by moving the split earlier and fitting everything on training data only.

MODELING

Three models, one shared pipeline

Each model is a scikit-learn Pipeline wrapping the same ColumnTransformer, tuned with 5-fold GridSearchCV. Next: each model, presented by the person who built it.

Support Vector Regression

Best Parameters

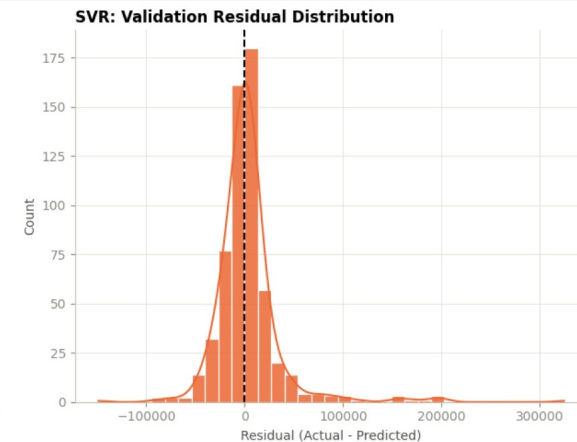
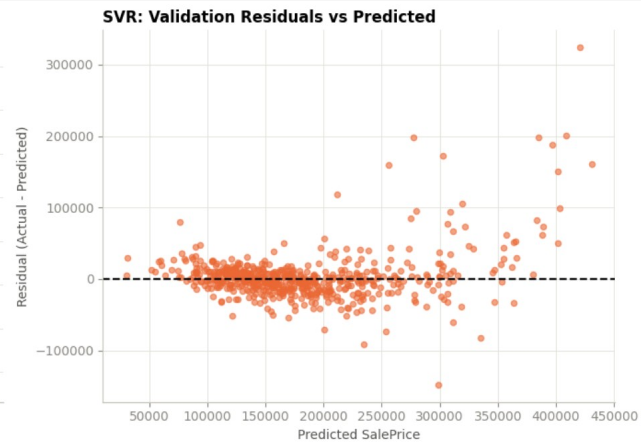
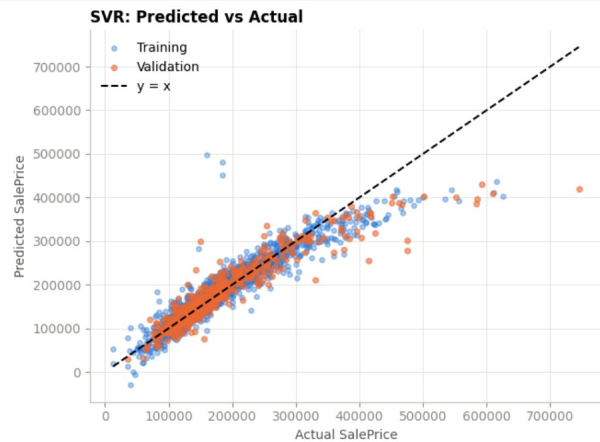
C = 5000, ε = 0.005

CV RMSE

\$29,748

Training R²

0.874



MODEL 2 · PRESENTED BY [TEAMMATE 1]

Linear Regression

Winner
Best model

Key technique

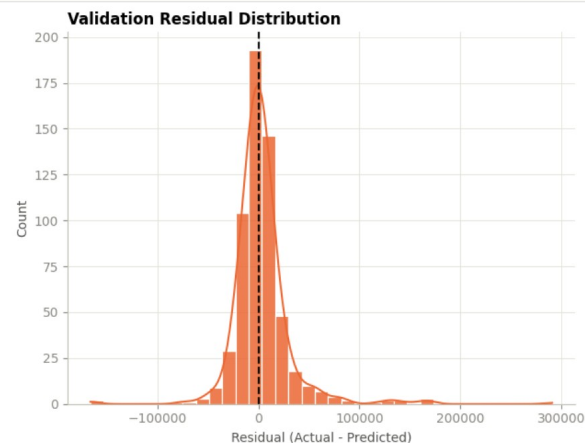
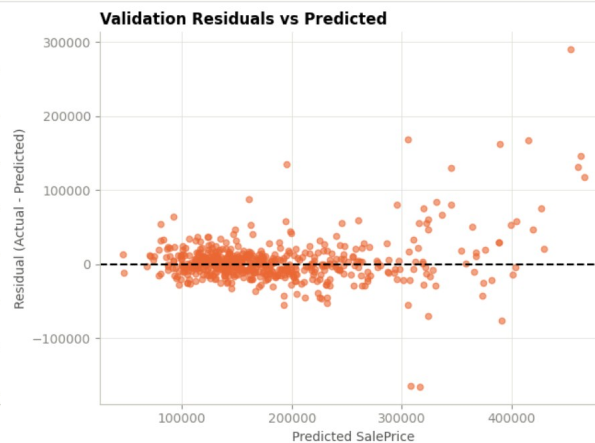
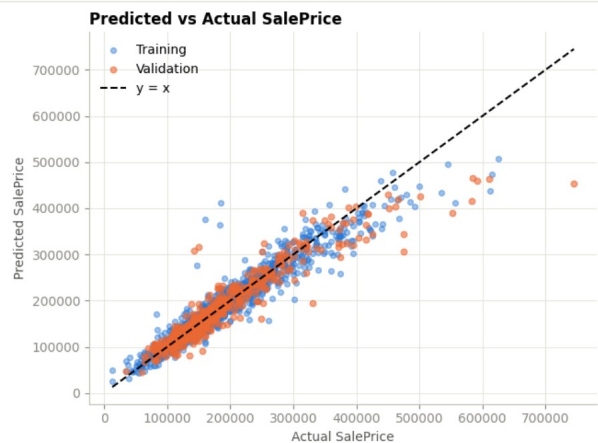
Log-transformed target

Validation RMSE

\$30,495

Validation R²

0.879



MODEL 3 · PRESENTED BY [TEAMMATE 2]

K-Nearest Neighbors

Best Parameters

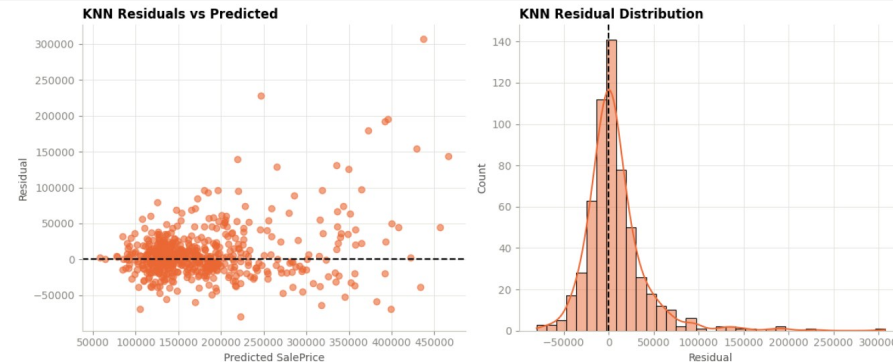
k=5, distance-weighted, Manhattan

Validation RMSE

\$36,221

Validation R^2

0.830

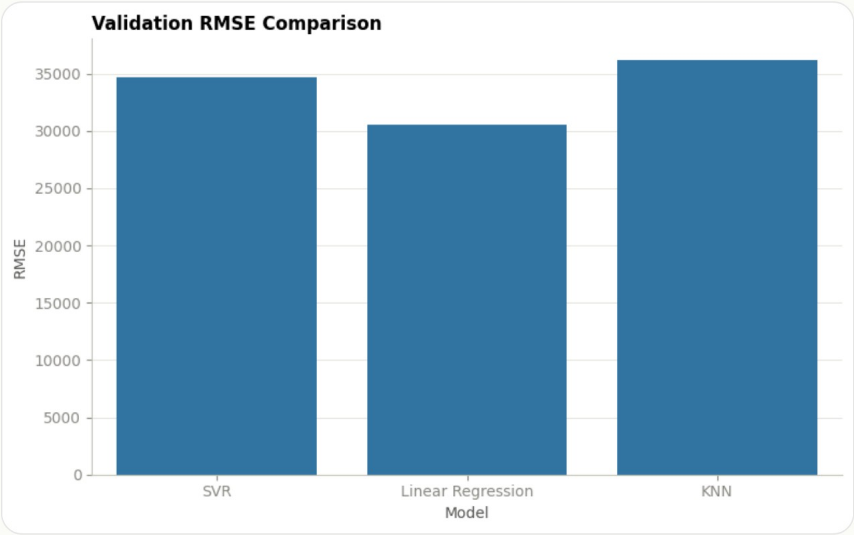


⚠ **Gotcha we caught:** with distance-weighting, KNN "predicts" its own training points perfectly (each is its own nearest neighbor at distance 0) — giving Train $R^2 = 1.0$. Not a bug: validation/test metrics are the only meaningful ones for this model.

RESULTS

Linear Regression wins on validation

Model	Val MAE	Val RMSE	Val R ²
Linear Regression	\$16,917	\$30,495	0.879
SVR	\$19,026	\$34,649	0.844
KNN	\$21,746	\$36,221	0.830



Final test-set RMSE (winner)

\$22,374

Test RMSE actually came in **lower** than validation this run (R^2 0.915 vs 0.879) — the reverse of the usual pattern. With a right-skewed target and ~586-row splits, RMSE is very sensitive to exactly which high-value homes land in validation vs. test; the relative ranking of the three models is what to trust, not the exact number on any one split.

CONCLUSION

Why the simplest model won, and what we'd do next



Data is close to linear

After Yeo-Johnson and log-transforms, price relates near-linearly to features — SVR's own grid search agreed, picking a linear kernel.



RMSE is split-sensitive

A skewed target means which high-value homes land in each split noticeably swings RMSE. A stratified split is a natural next step.



Merging pipelines needs care

Verifying every teammate's model shared the same fitted preprocessing caught a leakage bug and a consistency gap before submission.

Thank you — questions welcome.